Frank-Wolfe Algorithm: Description, Pseudocode, and Convergence Analysis

1 Algorithm Description

The algorithm iteratively updates parameters by solving a linear minimization subproblem over the constraint set and moving towards this solution along a line segment. It is designed for smooth convex optimization problems with closed convex and bounded constraint domains.

1.1 Algorithm Name

Frank-Wolfe Algorithm (Conditional Gradient Method)

1.2 Mathematical Formulation

Problem Statement: Minimize a smooth convex function $f(w)$ over a closed convex bounded constraint set $\mathcal{C} \subseteq \mathbb{R}^n$:

$$\min_{w \in \mathcal{C}} f(w)$$

Assumptions: - $f(w)$ is differentiable and convex. - $\mathcal{C}$ is closed, convex, and bounded.

Update Rules: At iteration t: 1. Compute the gradient $\nabla f(w_t)$. 2. Solve the linear minimization subproblem:

$$s_t = \arg\min_{s \in \mathcal{C}} \nabla f(w_t)^T s$$

3. Update parameters:

$$w_{t+1} = w_t + \eta_t (s_t - w_t)$$

where $\eta_t \in [0, 1]$ is the step size.

Hyperparameters: - **Step Size η_t:** Determines how far to move toward s_t. Options include: - Fixed: $\eta_t = \eta \in (0, 1)$. - Diminishing: $\eta_t = \frac{2}{t+2}$. - Line Search: $\eta_t = \arg\min_{\eta \in [0,1]} f(w_t + \eta(s_t - w_t))$. - **Constraint Set $\mathcal{C}$:** Must be closed, convex, and bounded.

Convergence Guarantee: For convex f and $\eta_t = \frac{2}{t+2}$, the Frank-Wolfe algorithm converges to the optimal value f^* with a sublinear rate $f(w_t) - f^* = O(1/t)$.

2 Pseudocode and Dry Run

2.1 Pseudocode

Algorithm 1 Frank-Wolfe Algorithm

Inputs: Objective function f, constraint set $\mathcal{C}$, step size rule η_t, maximum iterations T, tolerance ϵ.
Initialize: Choose $w_0 \in \mathcal{C}$.
for $t = 0$ to T **do**
    Compute gradient $\nabla f(w_t)$.
    Solve $s_t = \arg\min_{s \in \mathcal{C}} \nabla f(w_t)^T s$.
    Choose step size $\eta_t \in [0, 1]$.
    Update $w_{t+1} = w_t + \eta_t(s_t - w_t)$.
    Check termination:
    if $\|\nabla f(w_t)\| \leq \epsilon$ or $\|w_{t+1} - w_t\| \leq \epsilon$ or $t = T$ **then**
        Break
    end if
end for
Output: Final iterate w_T.

2.2 Dry Run Example

Objective Function:

$$f(w) = (w - 3)^2 \quad \text{(with constraint set } \mathcal{C} = [0, 6])$$

Hyperparameters: - Initialization: $w_0 = 0$ - Fixed step size: $\eta = 0.1$ - Minibatch size: 1 (deterministic gradient)

Iteration 1: 1. Compute gradient:

$$\nabla f(w_0) = 2(w_0 - 3) = 2(0 - 3) = -6$$

2. Solve linear subproblem:

$$s_0 = \arg\min_{s\in[0,6]} (-6)s = 6 \quad (\text{minimizes } -6s)$$

3. Update parameters:

$$w_1 = w_0 + 0.1(s_0 - w_0) = 0 + 0.1(6 - 0) = 0.6$$

4. Objective value:

$$f(w_1) = (0.6 - 3)^2 = 5.76$$

Iteration 2: 1. Compute gradient:

$$\nabla f(w_1) = 2(0.6 - 3) = -4.8$$

2. Solve linear subproblem:

$$s_1 = \arg\min_{s\in[0,6]} (-4.8)s = 6$$

3. Update parameters:

$$w_2 = w_1 + 0.1(s_1 - w_1) = 0.6 + 0.1(6 - 0.6) = 1.14$$

4. Objective value:

$$f(w_2) = (1.14 - 3)^2 = 3.4596$$

Iteration 3: 1. Compute gradient:

$$\nabla f(w_2) = 2(1.14 - 3) = -3.72$$

2. Solve linear subproblem:

$$s_2 = \arg\min_{s\in[0,6]} (-3.72)s = 6$$

3. Update parameters:

$$w_3 = w_2 + 0.1(s_2 - w_2) = 1.14 + 0.1(6 - 1.14) = 1.626$$

4. Objective value:

$$f(w_3) = (1.626 - 3)^2 = 1.8879$$

Summary of Iterations:

Iteration	w_t	$\nabla f(w_t)$	s_t	w_{t+1}	$f(w_{t+1})$
0	0	-6	6	0.6	5.76
1	0.6	-4.8	6	1.14	3.4596
2	1.14	-3.72	6	1.626	1.8879

Observation: With fixed $\eta = 0.1$, the algorithm moves toward $s_t = 6$ at each iteration, reducing $f(w)$ but converging slowly toward the optimal value $f^* = 0$. Proper step size selection (e.g., line search or diminishing rules) is critical for convergence to the true minimum at $w = 3$.

3 Convergence Theory

3.1 Assumptions

We consider the constrained optimization problem:

$$\min_{x \in \mathcal{C}} f(x)$$

where the following assumptions hold:

Assumption 1 (Constraint Set)**.** *$\mathcal{C} \subset \mathbb{R}^n$ is a closed, convex, and bounded set with finite diameter:*

$$D = \sup_{x,y \in \mathcal{C}} \|x - y\| < \infty.$$

Assumption 2 (Objective Function)**.** *$f : \mathbb{R}^n \to \mathbb{R}$ is convex and L-smooth, meaning:*

1. *(Convexity) $f(y) \geq f(x) + \nabla f(x)^\top (y - x)$ for all $x, y \in \mathcal{C}$;*
2. *(Lipschitz Gradient) $\|\nabla f(y) - \nabla f(x)\| \leq L\|y - x\|$ for all $x, y \in \mathcal{C}$.*

Assumption 3 (Algorithm Parameters)**.** *The algorithm uses step sizes $\gamma_t \in (0, 1]$ satisfying:*

$$\sum_{t=1}^{\infty} \gamma_t = \infty \quad \textit{and} \quad \sum_{t=1}^{\infty} \gamma_t^2 < \infty.$$

3.2 Main Convergence Theorem

Theorem 1. *Let $\{x_t\}$ be the sequence generated by the algorithm:*

$$x_{t+1} = x_t + \gamma_t (s_t - x_t),$$

where $s_t = \arg\min_{s \in \mathcal{C}} \nabla f(x_t)^\top s$. Under Assumptions 1-3, the suboptimality gap satisfies:

$$f(x_T) - f^* \leq \frac{2LD^2}{T + 2}, \quad \forall T \geq 1,$$

when $\gamma_t = \frac{2}{t+2}$. In particular, $\lim_{T \to \infty} f(x_T) = f^*$.

Proof. We use induction and smoothness:

$$f(x_{t+1}) \leq f(x_t) + \gamma_t \nabla f(x_t)^\top (s_t - x_t) + \frac{L\gamma_t^2}{2} D^2.$$

Define $h_t = f(x_t) - f^*$. By convexity, $\nabla f(x_t)^\top (s_t - x_t) \leq -h_t$. Substituting:

$$h_{t+1} \leq h_t - \gamma_t h_t + \frac{L\gamma_t^2}{2} D^2.$$

Choosing $\gamma_t = \frac{2}{t+2}$, we derive a recursive bound and show $h_T \leq \frac{2LD^2}{T+2}$ by induction. □

3.3 Theoretical Properties

Proposition 1 (Stability). *Let $\tilde{s}_t$ satisfy $\nabla f(x_t)^\top \tilde{s}_t \leq \min_{s \in \mathcal{C}} \nabla f(x_t)^\top s + \epsilon_t$. Then:*

$$f(x_T) - f^* = O\left(\frac{1}{T} + \frac{1}{T}\sum_{t=1}^{T-1} \epsilon_t\right).$$

Proposition 2 (Step Size Sensitivity). *Using exact line search ($\gamma_t = \arg\min_{\gamma \in [0,1]} f(x_t + \gamma(s_t - x_t))$) preserves $O(1/T)$ convergence. Constant step sizes $\gamma_t = \frac{1}{2L}$ yield convergence to a neighborhood of size $O(L^{-1})$.*

Proposition 3 (Comparison to Baselines).

1. *Projected Gradient Descent: Requires $O(1/T)$ iterations for $O(1/\sqrt{T})$ accuracy without smoothness and $O(1/T^2)$ with acceleration.*
2. *Frank-Wolfe: Avoids projections at the cost of slower rate; linear convergence possible for strongly convex objectives over polytopes.*

3.4 Discussion

The algorithm achieves an $O(1/T)$ convergence rate, matching the optimal non-asymptotic rate for first-order methods on smooth convex problems. Key advantages include:

- Computational efficiency per iteration via linear minimization;
- Sparse representation of iterates as convex combinations of extreme points.

Limitations include dependence on the squared diameter D^2 and inability to exploit strong convexity of f unless $\mathcal{C}$ is a polytope. For example, in matrix completion ($\mathcal{C}$ = nuclear norm ball), this method outperforms projected gradient descent when linear minimization (via SVD) is significantly cheaper than full projections.

4 Convergence Proofs

4.1 Preliminaries (Definitions and Lemmas)

4.1.1 Definitions

Definition 1 (Convex Function): A function $f : \mathbb{R}^n \to \mathbb{R}$ is convex if for all $x, y \in \mathbb{R}^n$ and $\lambda \in [0, 1]$:

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y).$$

Definition 2 (L-smooth Function): A differentiable function f is L-smooth if its gradient is L-Lipschitz continuous:

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\| \quad \forall x, y \in \mathbb{R}^n.$$

This implies the quadratic upper bound:

$$f(y) \leq f(x) + \nabla f(x)^\top (y - x) + \frac{L}{2}\|y - x\|^2.$$

Definition 3 (Diameter of Constraint Set): The constraint set $\mathcal{C} \subset \mathbb{R}^n$ has finite diameter:

$$D = \sup_{x,y \in \mathcal{C}} \|x - y\| < \infty.$$

4.1.2 Key Lemmas

Lemma 1 (Convexity Lower Bound): For any $x, y \in \mathcal{C}$:

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x).$$

Lemma 2 (L-smoothness Upper Bound): For any $x, y \in \mathcal{C}$:

$$f(y) \leq f(x) + \nabla f(x)^\top (y - x) + \frac{L}{2}\|y - x\|^2.$$

Lemma 3 (Linear Minimizer Property): Let $s_t = \arg\min_{s \in \mathcal{C}} \nabla f(x_t)^\top s$. Then:

$$\nabla f(x_t)^\top (s_t - x_t) \leq -h_t,$$

where $h_t = f(x_t) - f^*$.

Proof of Lemma 3: By convexity (Lemma 1), $f^* \geq f(x_t) + \nabla f(x_t)^\top (x^* - x_t)$, which rearranges to:

$$\nabla f(x_t)^\top (x^* - x_t) \leq -h_t.$$

Since s_t minimizes $\nabla f(x_t)^\top s$ over $\mathcal{C}$, we have $\nabla f(x_t)^\top s_t \leq \nabla f(x_t)^\top x^*$. Subtracting $\nabla f(x_t)^\top x_t$ from both sides gives:

$$\nabla f(x_t)^\top (s_t - x_t) \leq \nabla f(x_t)^\top (x^* - x_t) \leq -h_t.$$

4.2 Main Proof (Step-by-Step Derivation)

4.2.1 Algorithm Update Rule

The Frank-Wolfe algorithm generates iterates via:

$$x_{t+1} = x_t + \gamma_t(s_t - x_t),$$

where $s_t = \arg\min_{s \in \mathcal{C}} \nabla f(x_t)^\top s$.

4.2.2 Step 1: Apply Smoothness Inequality

Using Lemma 2 with $y = x_{t+1}$ and $x = x_t$:

$$f(x_{t+1}) \leq f(x_t) + \nabla f(x_t)^\top (x_{t+1} - x_t) + \frac{L}{2}\|x_{t+1} - x_t\|^2.$$

Substitute $x_{t+1} - x_t = \gamma_t(s_t - x_t)$:

$$f(x_{t+1}) \leq f(x_t) + \gamma_t \nabla f(x_t)^\top (s_t - x_t) + \frac{L\gamma_t^2}{2}\|s_t - x_t\|^2.$$

4.2.3 Step 2: Bound the Quadratic Term

Since $s_t, x_t \in \mathcal{C}$, we have $\|s_t - x_t\| \leq D$. Thus:

$$f(x_{t+1}) \leq f(x_t) + \gamma_t \nabla f(x_t)^\top (s_t - x_t) + \frac{L\gamma_t^2}{2} D^2.$$

4.2.4 Step 3: Use Lemma 3 to Bound the Inner Product

From Lemma 3, $\nabla f(x_t)^\top (s_t - x_t) \leq -h_t$. Substituting:

$$f(x_{t+1}) \leq f(x_t) - \gamma_t h_t + \frac{L\gamma_t^2 Kedrova\gamma_t^2}{2} D^2.$$

Subtract f^* from both sides:

$$h_{t+1} \leq h_t(1 - \gamma_t) + \frac{L\gamma_t^2}{2} D^2. \quad \text{(Equation 1)}$$

4.3 Rate Derivation (With Constants)

4.3.1 Step Size Choice: $\gamma_t = \frac{2}{t+2}$

We prove by induction that:

$$h_T \leq \frac{2LD^2}{T+2} \quad \forall T \geq 1.$$

Base Case ($T = 1$): Assume x_1 is obtained from x_0 via $\gamma_0 = \frac{2}{0+2} = 1$, so $x_1 = s_0$. Then:

$$h_1 = f(s_0) - f^*.$$

Using smoothness between s_0 and x^*:

$$f(s_0) \leq f(x^*) + \nabla f(x^*)^\top (s_0 - x^*) + \frac{L}{2}\|s_0 - x^*\|^2.$$

Since $f(x^*)$ is the minimum, $\nabla f(x^*)^\top (s_0 - x^*) \geq 0$, so:

$$h_1 \leq \frac{L}{2} D^2 \leq \frac{2LD^2}{3} \quad (\text{since } D^2 \leq D^2).$$

Inductive Step: Assume $h_t \leq \frac{2LD^2}{t+2}$. From Equation 1 with $\gamma_t = \frac{2}{t+2}$:

$$h_{t+1} \leq h_t \left(\frac{t}{t+2}\right) + \frac{2LD^2}{(t+2)^2}.$$

Apply the induction hypothesis:

$$h_{t+1} \leq \frac{2LD^2}{t+2} \cdot \frac{t}{t+2} + \frac{2LD^2}{(t+2)^2} = \frac{2LD^2(t+1)}{(t+2)^2}.$$

We need to show:

$$\frac{2LD^2(t+1)}{(t+2)^2} \leq \frac{2LD^2}{t+3}.$$

Cancel $2LD^2$ and cross-multiply:

$$(t+1)(t+3) \leq (t+2)^2.$$

Expand both sides:

$$t^2 + 4t + 3 \leq t^2 + 4t + 4.$$

This holds, completing the induction.

4.3.2 Final Bound

By induction, for all $T \geq 1$:

$$h_T \leq \frac{2LD^2}{T+2}.$$

4.4 Special Cases

4.4.1 Approximate Linear Minimization

If $\tilde{s}_t$ satisfies $\nabla f(x_t)^\top \tilde{s}_t \leq \min_{s \in \mathcal{C}} \nabla f(x_t)^\top s + \epsilon_t$, then:

$$\nabla f(x_t)^\top (\tilde{s}_t - x_t) \leq -h_t + \epsilon_t.$$

Substituting into Equation 1:

$$h_{t+1} \leq h_t(1 - \gamma_t) + \gamma_t \epsilon_t + \frac{L\gamma_t^2}{2} D^2.$$

With $\gamma_t = \frac{2}{t+2}$, this leads to:

$$h_T = O\left(\frac{1}{T} + \frac{1}{T}\sum_{t=1}^{T-1} \epsilon_t\right).$$

4.4.2 Step Size Sensitivity

- **Exact Line Search:** Choosing $\gamma_t = \arg\min_{\gamma \in [0,1]} f(x_t + \gamma(s_t - x_t))$ preserves the $O(1/T)$ rate because the decrease is at least as good as with $\gamma_t = \frac{2}{t+2}$.

- **Constant Step Size $\gamma_t = \frac{1}{2L}$:** Substituting into Equation 1:

$$h_{t+1} \leq h_t\left(1 - \frac{1}{2L}\right) + \frac{D^2}{8L}.$$

This recursion converges to a neighborhood of size $O\left(\frac{D^2}{L}\right)$.

4.4.3 Comparison to Baselines

- **Projected Gradient Descent (PGD):** Requires $O(1/T)$ iterations for $O(1/\sqrt{T})$ accuracy without smoothness and $O(1/T^2)$ with acceleration. Each iteration requires a projection onto $\mathcal{C}$, which is computationally expensive for complex $\mathcal{C}$.

- **Frank-Wolfe:** Avoids projections at the cost of a slower $O(1/T)$ rate. Linear minimization is cheaper for polytopes (via vertex search) and nuclear norm balls (via SVD).

4.4.4 Example: Matrix Completion

In matrix completion, $\mathcal{C}$ is the nuclear norm ball. Linear minimization corresponds to computing the leading singular vector (via power method), which is significantly cheaper than projecting onto the nuclear norm ball (requires full SVD). Thus, Frank-Wolfe outperforms PGD in this setting.

4.5 Discussion

The Frank-Wolfe algorithm achieves an $O(1/T)$ convergence rate for smooth convex optimization over closed, convex, bounded domains. Its key advantages are computational efficiency per iteration via linear minimization and sparse representation of iterates as convex combinations of extreme points. However, its rate depends on the squared diameter D^2 and does not exploit strong convexity of f unless $\mathcal{C}$ is a polytope. This makes it particularly effective for problems like matrix completion where linear minimization is inexpensive.